

```

In [1]: library(Seurat)
# library(SeuratDisk)

library(reticulate)
library(anndata)

library(ggplot2)
library(ggpubr)
library(pheatmap)
library(dplyr)
library(tidyr)
library(RColorBrewer)
library(clustree)
library(cowplot)
library(hexbin)
library(repr)
library(reshape2)
library(scico)
#library(carto)
options(repr.plot.width=10, repr.plot.height=8)

library(UpSetR)
library(grid)

library(PRRoc)
library(Matrix)

library(MASS)

getwd()

dataset_id <- "simulated_mm_RA"
genome_id <- "mm10"
samples <- c("old", "young") #all

for (sample in samples) {
  dir.create(paste0("figures_", dataset_id, "_", sample))
}

colorTools <- c( # "MATES"="#F98A7B",
  "STARsolo_TE" = "#FFBC81",
  "STARsolo_TE_EM" = "#F3E088",
  "SoloTE_unique" = "#A4DD9B",
  "SoloTE_thr2" = "#6FC69D",
  "SoloTE_thr1" = "#4BB2BB",
  "SoloTE_thr0" = "#3989BF",
  "Stellarscope" = "#4F5D93",
  "simulated" = "grey70"
)

colorSamples <- c("old"="#7A646A",
  "young"="#A3C6E3",
  "all"="#8190a9")

thrMinCells <- 500 * 0.05

theme_set(
  theme_minimal(base_size = 20)
)

```

Loading required package: SeuratObject

Loading required package: sp

'SeuratObject' was built under R 4.3.3 but the current version is 4.4.1; it is recommended that you reinstall 'SeuratObject' as the ABI for R may have changed

'SeuratObject' was built with package 'Matrix' 1.6.4 but the current version is 1.7.5; it is recommended that you reinstall 'SeuratObject' as the ABI for 'Matrix' may have changed

Attaching package: 'SeuratObject'

The following objects are masked from 'package:base':

intersect, t

Attaching package: 'anndata'

The following object is masked from 'package:SeuratObject':

Layers

Warning message:

"package 'dplyr' was built under R version 4.4.3"

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Warning message:

"package 'tidyr' was built under R version 4.4.3"

Warning message:

"package 'RColorBrewer' was built under R version 4.4.3"

Loading required package: ggraph

Attaching package: 'ggraph'

The following object is masked from 'package:sp':

geometry

Warning message:

"package 'cowplot' was built under R version 4.4.3"

Attaching package: 'cowplot'

The following object is masked from 'package:ggraph':

get_legend

Warning message:

"package 'reshape2' was built under R version 4.4.3"

Attaching package: 'reshape2'

The following object is masked from 'package:tidyr':

smiths

Loading required package: rlang

Warning message:

"package 'rlang' was built under R version 4.4.3"

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Warning message:

"package 'MASS' was built under R version 4.4.3"

Attaching package: 'MASS'

The following object is masked from 'package:dplyr':

```
select
```

```
"/mnt/TEdata/TEbenchmarking/Rscripts'
```

```
Warning message in dir.create(paste0("figures_", dataset_id, "_", sample)):  
"figures_simulated_mm_RA_old' already exists"  
Warning message in dir.create(paste0("figures_", dataset_id, "_", sample)):  
"figures_simulated_mm_RA_young' already exists"
```

```
In [2]: # import workspace  
load("workspaces/00_evaluation_objectCreation.Rdata")
```

```
In [3]: # function to expand matrix to certain list of rows and columns  
  
expand_numeric_matrix <- function(M, rows, cols) {  
  out <- Matrix(0, nrow = length(rows), ncol = length(cols), sparse = TRUE)  
  rownames(out) <- rows  
  colnames(out) <- cols  
  
  out[rownames(M), colnames(M)] <- M  
  out  
}
```

Expression of FPs

Raw counts

```
In [4]: # get all FP counts = counts that are detected in a cell where a locus is not expressed  
  
layer <- "counts"  
  
FPcountList <- list()  
TPcountList <- list()  
  
for(sample in samples){  
  
  splatter_obj <- splatter_objs[[sample]]  
  
  FPcountList[[sample]] <- NULL  
  
  for(tool in names(objList)){  
  
    obj <- objList[[tool]][[sample]]  
  
    splatterMat <- GetAssayData(splatter_obj, layer = layer)  
    objMat <- GetAssayData(obj, layer = layer)  
  
    # 1. Get full union of rows and columns  
    all_rows <- union(rownames(splatterMat), rownames(objMat))  
    all_cols <- union(colnames(splatterMat), colnames(objMat))  
  
    splatterMat_exp <- expand_numeric_matrix(splatterMat, all_rows, all_cols)  
    objMat_exp <- expand_numeric_matrix(objMat, all_rows, all_cols)  
  
    # Compute False Positives  
    FP_mask <- (objMat_exp > 0) & (splatterMat_exp == 0)  
  
    ij <- which(FP_mask, arr.ind = TRUE)  
  
    fp_df <- data.frame(  
      feature = rownames(objMat_exp)[ij[,1]],  
      cell    = colnames(objMat_exp)[ij[,2]],  
      value   = objMat_exp[ij],  
      row.names = NULL  
    )  
  
    colnames(fp_df) <- c("feature", "cell", "value")  
    fp_df$tool <- tool  
    fp_df$age <- sample  
    FPcountList[[sample]][[tool]] <- fp_df  
  
    # Compute True Positives  
    TP_mask <- (objMat_exp > 0) & (splatterMat_exp > 0)  
    ij <- which(TP_mask, arr.ind = TRUE)  
    tp_df <- data.frame(  
      feature = rownames(objMat_exp)[ij[,1]],  
      cell    = colnames(objMat_exp)[ij[,2]],  
      value   = objMat_exp[ij],  
      row.names = NULL  
    )  
  }  
}
```

```

    )
    colnames(tp_df) <- c("feature", "cell", "value")
    tp_df$tool <- tool
    tp_df$age <- sample
    TPcountList[[sample]][[tool]] <- tp_df
  }
}

```

```

In [6]: FPcountDf <- list()
TPcountDf <- list()
FPvsTPcountDf <- list()
for(sample in samples){
  FPcountDf[[sample]] <- do.call(rbind, FPcountList[[sample]])
  FPcountDf[[sample]]$type <- "FP"
  TPcountDf[[sample]] <- do.call(rbind, TPcountList[[sample]])
  TPcountDf[[sample]]$type <- "TP"
  FPvsTPcountDf[[sample]] <- rbind(TPcountDf[[sample]], FPcountDf[[sample]])
}

```

```

In [7]: FPcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
FPcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

FPcountDf[["young"]]$FPOverAvgSim <- FPcountDf[["young"]]$value / FPcountDf[["young"]]$avgExpr

FPcountDf[["young"]]$normOverSim <- FPcountDf[["young"]]$value / FPcountDf[["young"]]$totCounts * 10000

```

```

In [8]: FPvsTPcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
FPvsTPcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

FPvsTPcountDf[["young"]]$FPOverAvgSim <- FPvsTPcountDf[["young"]]$value / FPvsTPcountDf[["young"]]$avgExpr

FPvsTPcountDf[["young"]]$normOverSim <- FPvsTPcountDf[["young"]]$value / FPvsTPcountDf[["young"]]$totCounts * 10000

```

```

In [9]: TPcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
TPcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

TPcountDf[["young"]]$FPOverAvgSim <- TPcountDf[["young"]]$value / TPcountDf[["young"]]$avgExpr

TPcountDf[["young"]]$normOverSim <- TPcountDf[["young"]]$value / TPcountDf[["young"]]$totCounts * 10000

```

```

In [10]: FPcountDf[["young"]] %>%
  group_by(tool) %>%
  summarise(
    median_value = median(value, na.rm = TRUE),
    median_avgExpr = median(avgExpr, na.rm = TRUE),
    median_FPOverAvgSim = median(FPOverAvgSim, na.rm = TRUE),
    median_normOverSim = median(normOverSim, na.rm = TRUE)
  )

```

A tibble: 7 × 5

tool	median_value	median_avgExpr	median_FPOverAvgSim	median_normOverSim
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
STARsolo_TE	1.00000	6	0.1666667	0.1789741
STARsolo_TE_EM	1.11811	6	0.2000000	0.2008469
SoloTE_thr0	1.00000	6	0.2000000	0.1921525
SoloTE_thr1	1.00000	6	0.1666667	0.1817389
SoloTE_thr2	1.00000	6	0.1666667	0.1793979
SoloTE_unique	1.00000	6	0.1666667	0.1776641
Stellarscope	1.00000	6	0.2000000	0.2051892

```

In [11]: TPcountDf[["young"]] %>%
  group_by(tool) %>%
  summarise(
    median_value = median(value, na.rm = TRUE),
    median_avgExpr = median(avgExpr, na.rm = TRUE),
    median_FPOverAvgSim = median(FPOverAvgSim, na.rm = TRUE),
    median_normOverSim = median(normOverSim, na.rm = TRUE)
  )

```

A tibble: 7 × 5

tool	median_value	median_avgExpr	median_FPoverAvgSim	median_normOverSim
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
STARsolo_TE	4.000000	6	0.6666667	0.6707898
STARsolo_TE_EM	6.004005	6	1.0000000	1.0213417
SoloTE_thr0	4.000000	6	0.6666667	0.6732949
SoloTE_thr1	3.000000	6	0.5000000	0.5179290
SoloTE_thr2	3.000000	6	0.5000000	0.5129731
SoloTE_unique	3.000000	6	0.5000000	0.4878525
Stellarscope	6.000000	6	1.0000000	1.0382958

```
In [12]: options(repr.plot.width=7, repr.plot.height=4.5)

#for(sample in samples){
  sample <- "young"
  #reorder tools
  FPcountDf[[sample]]$tool <- factor(FPcountDf[[sample]]$tool, levels=rev(names(colorTools)))

  show(ggplot(FPcountDf[[sample]], aes(y=tool, x=value, fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Raw counts of FP ", sample, " TEs")) +
    coord_trans(x = "sqrt") +
    guides(fill="none") +
    theme(text=element_text(size=20),
    # plot.title = element_text(size=18, hjust=0.5),
    # plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=10),
    # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    #panel.spacing = unit(1, "lines")

    ))

  show(ggplot(FPcountDf[[sample]], aes(y=tool, x=log2(value+1), fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Raw counts of FP ", sample, " TEs")) +
    guides(fill="none") +
    xlab("log2 count") +
    theme(text=element_text(size=20),
    # plot.title = element_text(size=18, hjust=0.5),
    # plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=10),
    # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    #panel.spacing = unit(1, "lines")

    ))

  show(ggplot(FPcountDf[[sample]], aes(y=tool, x=normOverSim, fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Raw FP count / tot counts of ", sample, " TEs")) +
    guides(fill="none") +
    coord_trans(x = "sqrt") +
    theme(text=element_text(size=20),
    # plot.title = element_text(size=18, hjust=0.5),
    # plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=18),
    # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    #panel.spacing = unit(1, "lines")

    ))

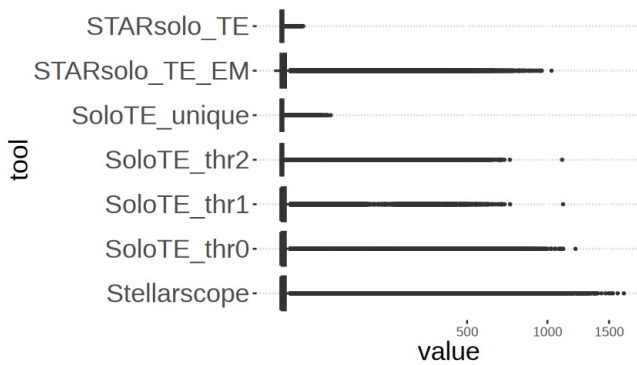
  ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FP_expression.pdf"), device="pdf", width=7, height=4.5)
  ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FP_expression.png"), device="png", width=7, height=4.5)
#}
```

Warning message:

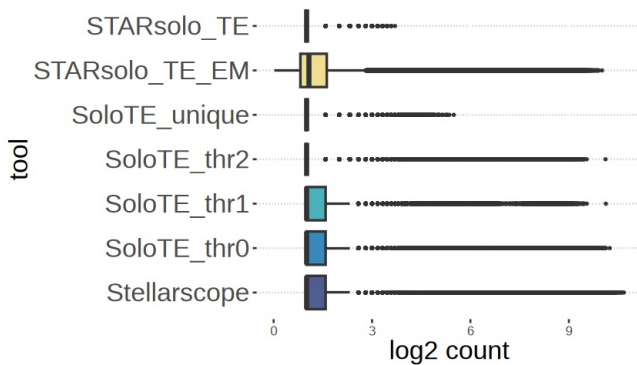
“`coord\_trans()`” was deprecated in ggplot2 4.0.0.

i Please use `coord\_transform()` instead.”

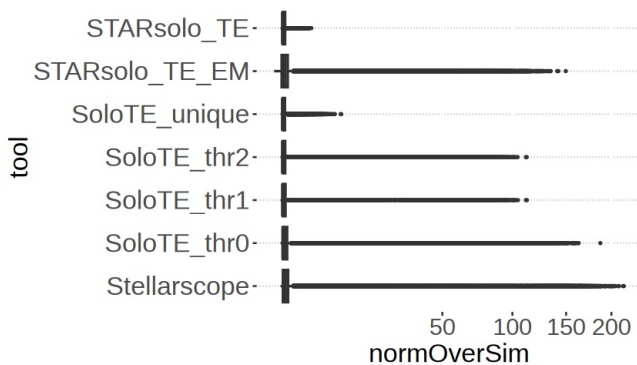
Raw counts of FP young TE:



Raw counts of FP young TE:



Raw FP count / tot counts of



```
In [29]: options(repr.plot.width=8, repr.plot.height=6.5)
library(ggtrastr)

#for(sample in samples){
  sample <- "young"
  #reorder tools

# Ensure ggtrastr uses the right engine
options(ggtrastr.default.device = "ragg")

sample <- "young"

# Reorder tools
FPvsTPcountDf[[sample]]$tool <- factor(FPvsTPcountDf[[sample]]$tool, levels=rev(names(colorTools)))

p <- ggplot(FPvsTPcountDf[[sample]], aes(y=tool, x=normOverSim, fill=type)) +
  # Wrap standard geom_boxplot in rasterise() to drop file size securely
  rasterise(
    geom_boxplot(outlier.size = 0.4, outlier.alpha = 0.3, linewidth=0.7, alpha=0.6),
    dpi = 300 # Adjust DPI if you need higher/lower print resolution
  ) +
  scale_fill_viridis_d() +
  # Keep your custom transformations and limits intact
  xlim(c(0, max(FPvsTPcountDf[[sample]]$normOverSim))) +
  coord_trans(x = "sqrt") +
  theme_pubclean() +
  ggtitle(paste0("FP vs TP counts of ", sample, " TEs"),
    subtitle="Normalized by tot simulated counts") +
  xlab("estimated / tot simulated counts") +
  theme(
    text = element_text(size=20),
    plot.title = element_text(size=20, hjust=0.5),
    plot.subtitle = element_text(size=20, hjust=0.5),
```

```

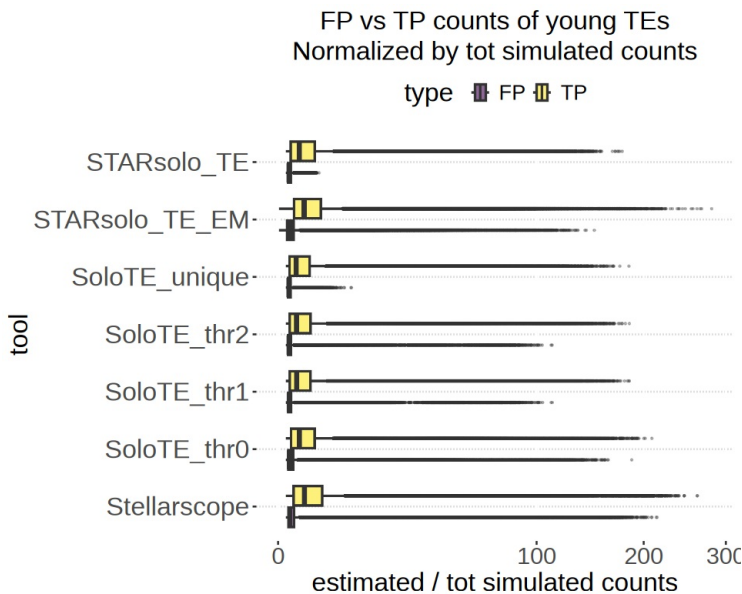
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=18)
  )

print(p)

ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FP_vs TP_expression.pdf"), device="pdf", width=8, height=4.5)
ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FP_vs TP_expression.pdf"), device="pdf", width=8, height=4.5)

#}

```



```

In [17]: df_cell_avg <- FPvsTPcountDf[[sample]] %>%
  group_by(tool, cell, type) %>%
  summarise(
    avg_count = mean(value, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  pivot_wider(
    names_from = type,
    values_from = avg_count
  ) %>%
  filter(!is.na(TP), !is.na(FP)) %>% # both must exist at cell level
  mutate(
    TP_FP_avg_ratio = TP / FP,
    log2_FC = log2((TP + 1e-6) / (FP + 1e-6))
  )

sample <- "young"
#reorder tools
df_cell_avg$tool <- factor(df_cell_avg$tool, levels=rev(names(colorTools)))

options(repr.plot.width=8, repr.plot.height=4.5)

show(ggplot(df_cell_avg, aes(y=tool, x=TP_FP_avg_ratio, fill=tool)) +
  geom_boxplot(outlier.size=0.5, linewidth=0.7, alpha = 0.6) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Fold-change of TP vs FP counts of ", sample, " TEs"), subtitle="per cell") +
  #coord_trans(x = "sqrt") +
  guides(fill="none") +
  theme(text=element_text(size=20),
    plot.title = element_text(size=20, hjust=0.5),
    plot.subtitle = element_text(size=20, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=18)
  ))
ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FoldChange_TP_vs_FP_expression.pdf"), device="pdf", width=8, height=4.5)

#violin
show(ggplot(df_cell_avg, aes(y=tool, x=TP_FP_avg_ratio, fill=tool)) +
  geom_violin(linewidth=0.7, alpha=0.6, trim = TRUE,
    draw_quantiles = c(0.25, 0.75)) +
  stat_summary(fun = median, geom = "point", shape = 18, size = 2, color = "black") +
  scale_fill_manual(values=colorTools) +
  scale_x_continuous(limits = c(0, NA), expand = expansion(mult = c(0, 0.05))) +
  theme_pubclean() + ggtitle(paste0("Fold-change of TP vs FP counts of ", sample), subtitle="TEs per cell") +
  guides(fill="none") +
  theme(text=element_text(size=20),
    plot.title = element_text(size=20, hjust=0.5),
    plot.subtitle = element_text(size=20, hjust=0.5),
    axis.text.y = element_text(size=20),

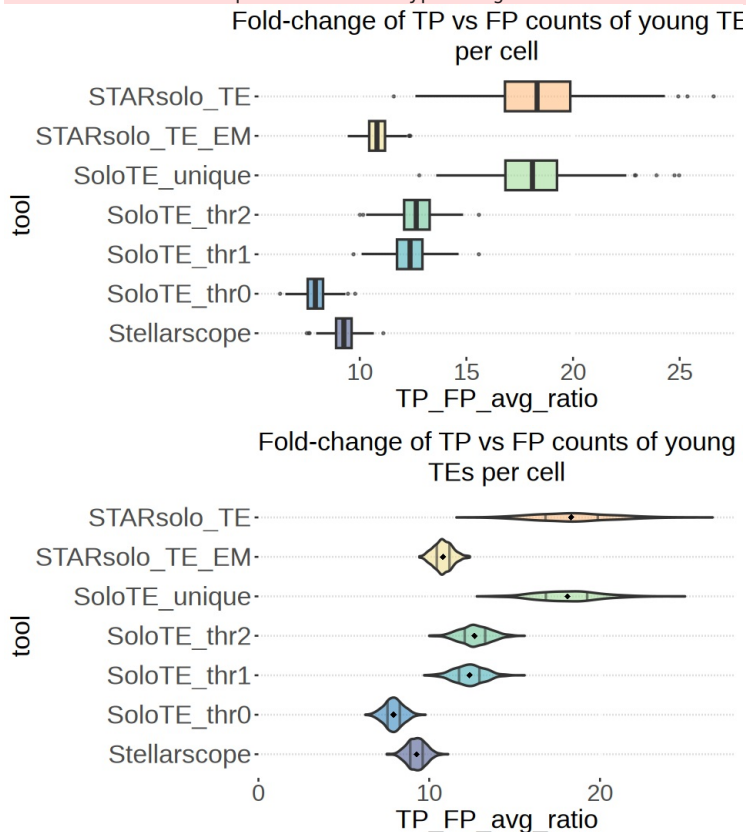
```

```
axis.text.x = element_text(size=18)
))
ggsave(paste0("figures_", dataset_id, "_", sample, "/violin_FoldChange_TP_vs_FP_expression.pdf"),
device="pdf", width=8, height=4.5)
```

Warning message:

"The `draw\_quantiles` argument of `geom\_violin()` is deprecated as of ggplot2 4.0.0.

Please use the `quantiles.linetype` argument instead."



Normalized counts

```
In [ ]: # get all FP counts = counts that are detected in a cell where a locus is not expressed
```

```
layer <- "data"

FPcountList <- list()

for(sample in samples){

  splatter_obj <- splatter_objs[[sample]]

  FPcountList[[sample]] <- NULL

  for(tool in names(objList)){

    obj <- objList[[tool]][[sample]]

    splatterMat <- GetAssayData(splatter_obj, layer = layer)
    objMat <- GetAssayData(obj, layer = layer)

    library(Matrix)

    # 1. Get full union of rows and columns
    all_rows <- union(rownames(splatterMat), rownames(objMat))
    all_cols <- union(colnames(splatterMat), colnames(objMat))

    splatterMat_exp <- expand_numeric_matrix(splatterMat, all_rows, all_cols)
    objMat_exp <- expand_numeric_matrix(objMat, all_rows, all_cols)

    # Compute False Positives
    FP_mask <- (objMat_exp > 0) & (splatterMat_exp == 0)

    ij <- which(FP_mask, arr.ind = TRUE)

    fp_df <- data.frame(
      feature = rownames(objMat_exp)[ij[,1]],
      cell = colnames(objMat_exp)[ij[,2]],
      value = objMat_exp[ij],
```



```

        row.names = NULL
    )

    colnames(fp_df) <- c("feature", "cell", "value")
    fp_df$tool <- tool
    fp_df$age <- sample
    FPcountList[[sample]][[tool]] <- fp_df
}
}

```

```

In [ ]: FPcountDf <- list()

for(sample in samples){
  FPcountDf[[sample]] <- do.call(rbind, FPcountList[[sample]])
}

```

```

In [ ]: head(FPcountDf[["young"]])

```

A data.frame: 6 × 5

	feature	cell	value	tool	age
	<chr>	<chr>	<dbl>	<chr>	<chr>
STARsolo_TE.1	L1Md-T-dup9335	AAACCTGCAGCCTATA	0.7258269	STARsolo_TE	young
STARsolo_TE.2	L1Md-A-dup7620	AAACCTGCAGCCTATA	0.4273698	STARsolo_TE	young
STARsolo_TE.3	IAPEz-int-dup2479	AAACCTGCAGCCTATA	0.4273698	STARsolo_TE	young
STARsolo_TE.4	IAPEz-int-dup379	AAACCTGCAGCCTATA	0.4273698	STARsolo_TE	young
STARsolo_TE.5	MERVL-int-dup2483	AAACCTGCAGCCTATA	0.4273698	STARsolo_TE	young
STARsolo_TE.6	MERVL-int-dup2478	AAACCTGCAGCCTATA	0.7258269	STARsolo_TE	young

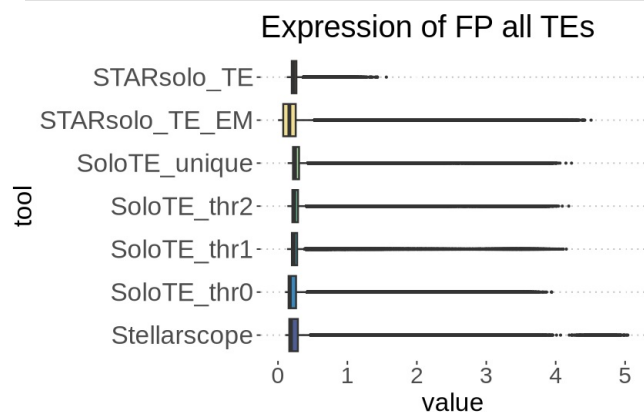
```

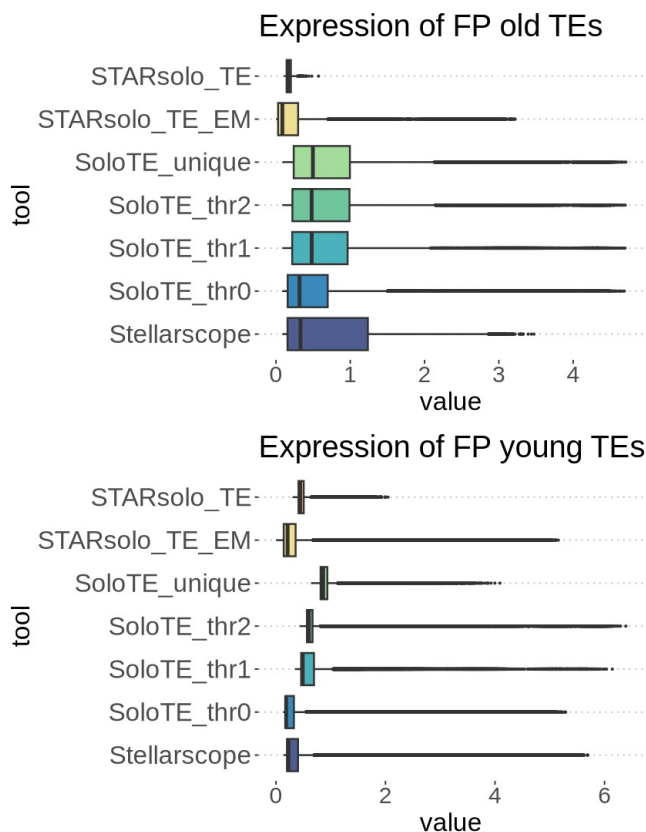
In [ ]: options(repr.plot.width=7, repr.plot.height=4.5)

for(sample in samples){
  #reorder tools
  FPcountDf[[sample]]$tool <- factor(FPcountDf[[sample]]$tool, levels=rev(names(colorTools)))

  show(ggplot(FPcountDf[[sample]], aes(y=tool, x=value, fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Expression of FP ", sample, " TEs")) +
    guides(fill="none") +
    theme(text=element_text(size=20),
          # plot.title = element_text(size=18, hjust=0.5),
          # plot.subtitle = element_text(size=18, hjust=0.5),
          axis.text.y = element_text(size=20),
          axis.text.x = element_text(size=18),
          # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
          #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
          #panel.spacing = unit(1, "lines")
    ))
  ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FP_expression.pdf"), device="pdf", width=7, height=4.5)
}

```





```
In [ ]: options(repr.plot.width=7, repr.plot.height=4.5)

sample <- "young"
#reorder tools
FPcountDf[[sample]]$tool <- factor(FPcountDf[[sample]]$tool, levels=rev(names(colorTools)))

ggplot(FPcountDf[[sample]], aes(y=tool, x=value, fill=tool)) +
  geom_boxplot(outlier.size=0.5, linewidth=0.7) +
  scale_fill_manual(values=colorTools) +
  #stat_compare_means(method = "wilcox.test") +
  theme_pubclean() + ggtitle(paste0("Expression of FP ", sample, " TEs")) +
  guides(fill="none") +
  theme(text=element_text(size=20),
        # plot.title = element_text(size=18, hjust=0.5),
        # plot.subtitle = element_text(size=18, hjust=0.5),
        axis.text.y = element_text(size=20),
        axis.text.x = element_text(size=18),
        # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
        #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
        #panel.spacing = unit(1, "lines"))

# ggsave(paste0("figures ", dataset_id, "_", sample, "/boxplot_FP_expression_wilcoxon.pdf")),
# device="pdf", width=7, height=4.5)
```

Saving 6.67 x 6.67 in image

```
In [ ]: # #If FP is not normally distributed (most common for counts)
# #Use Kruskal-Wallis (nonparametric ANOVA):
# df_fp_young <- FPcountDf[["young"]]

# # Ensure numeric
# df_fp_young$value <- as.numeric(df_fp_young$value)
# df_fp_young$tool <- as.factor(df_fp_young$tool)

# stats::kruskal.test(value ~ tool, data = as.data.frame(df_fp_young))
```

Kruskal-Wallis rank sum test

data: value by tool
Kruskal-Wallis chi-squared = 6084634, df = 6, p-value < 2.2e-16

The test tells us that there's significant difference between the tools, but not which ones

```
In [ ]: pairwise.wilcox.test(df_fp_young$value, df_fp_young$tool,
                           p.adjust.method = "BH")
```

Pairwise comparisons using Wilcoxon rank sum test with continuity correction

data: df_fp_young\$value and df_fp_young\$tool

	Stellarscope	SoloTE_thr0	SoloTE_thr1	SoloTE_thr2	SoloTE_unique
SoloTE_thr0	<2e-16	-	-	-	-
SoloTE_thr1	<2e-16	<2e-16	-	-	-
SoloTE_thr2	<2e-16	<2e-16	<2e-16	-	-
SoloTE_unique	<2e-16	<2e-16	<2e-16	<2e-16	-
STARsolo_TE_EM	<2e-16	<2e-16	<2e-16	<2e-16	<2e-16
STARsolo_TE	<2e-16	<2e-16	<2e-16	<2e-16	<2e-16

STARsolo_TE_EM

SoloTE_thr0	-
SoloTE_thr1	-
SoloTE_thr2	-
SoloTE_unique	-
STARsolo_TE_EM	-
STARsolo_TE	<2e-16

P value adjustment method: BH

Every comparison is significant

confrontare con distribuzione espressione TPs

avg FP expr / avg TP expr

raw expr FP / raw expr simulated matrix

```
In [ ]: #If FP is not normally distributed (most common for counts)
# use Kruskal-Wallis (nonparametric ANOVA):
df_old <- FPcountDf[["old"]]

# # Ensure numeric
# df_old$value <- as.numeric(df_old$value)
# df_old$tool <- as.factor(df_old$tool)

stats::kruskal.test(value ~ tool, data = as.data.frame(df_old))
```

Kruskal-Wallis rank sum test

data: value by tool

Kruskal-Wallis chi-squared = 34441, df = 6, p-value < 2.2e-16

Expression of FNs

Raw counts

```
In [18]: # get all FN counts = counts that are NOT detected in a cell where a locus IS expressed
```

```
layer <- "counts"

FNcountList <- list()
TPcountList <- list()

for(sample in samples){

  splatter_obj <- splatter_objs[[sample]]

  FNcountList[[sample]] <- NULL

  for(tool in names(objList)){

    obj <- objList[[tool]][[sample]]

    splatterMat <- GetAssayData(splatter_obj, layer = layer)
    objMat <- GetAssayData(obj, layer = layer)

    # avg raw expression per cell
    # avgExprCells <- colMeans(splatterMat)

    # avg raw expression per cell
    # totCountsCells <- colSums(splatterMat)

    # 1. Get full union of rows and columns
```

```

all_rows <- union(rownames(splatterMat), rownames(objMat))
all_cols <- union(colnames(splatterMat), colnames(objMat))

splatterMat_exp <- expand_numeric_matrix(splatterMat, all_rows, all_cols)
objMat_exp <- expand_numeric_matrix(objMat, all_rows, all_cols)

# Compute False Positives
FN_mask <- (objMat_exp == 0) & (splatterMat_exp > 0)

ij <- which(FN_mask, arr.ind = TRUE)

fn_df <- data.frame(
  feature = rownames(splatterMat_exp)[ij[,1]],
  cell    = colnames(splatterMat_exp)[ij[,2]],
  value   = splatterMat_exp[ij],
  row.names = NULL
)

colnames(fn_df) <- c("feature", "cell", "value")
fn_df$tool <- tool
fn_df$age <- sample
FNcountList[[sample]][[tool]] <- fn_df

# Compute True Positives
TP_mask <- (objMat_exp > 0) & (splatterMat_exp > 0)
ij <- which(TP_mask, arr.ind = TRUE)
tp_df <- data.frame(
  feature = rownames(splatterMat_exp)[ij[,1]],
  cell    = colnames(splatterMat_exp)[ij[,2]],
  value   = splatterMat_exp[ij],
  row.names = NULL
)

colnames(tp_df) <- c("feature", "cell", "value")
tp_df$tool <- tool
tp_df$age <- sample
TPcountList[[sample]][[tool]] <- tp_df
}
}

```

```

In [19]: FNcountDf <- list()
TPcountDf <- list()
FNvsTPcountDf <- list()
for(sample in samples){
  FNcountDf[[sample]] <- do.call(rbind, FNcountList[[sample]])
  FNcountDf[[sample]]$type <- "FN"
  TPcountDf[[sample]] <- do.call(rbind, TPcountList[[sample]])
  TPcountDf[[sample]]$type <- "TP"
  FNvsTPcountDf[[sample]] <- rbind(TPcountDf[[sample]], FNcountDf[[sample]])
}

```

```

In [20]: FNcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
FNcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

FNcountDf[["young"]]$FNoverAvgSim <- FNcountDf[["young"]]$value / FNcountDf[["young"]]$avgExpr

FNcountDf[["young"]]$normOverSim <- FNcountDf[["young"]]$value / FNcountDf[["young"]]$totCounts * 10000

```

```

In [21]: FNvsTPcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
FNvsTPcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

FNvsTPcountDf[["young"]]$FNoverAvgSim <- FNvsTPcountDf[["young"]]$value / FNvsTPcountDf[["young"]]$avgExpr

FNvsTPcountDf[["young"]]$normOverSim <- FNvsTPcountDf[["young"]]$value / FNvsTPcountDf[["young"]]$totCounts * 10000

```

```

In [22]: TPcountDf[["young"]]$avgExpr <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=
TPcountDf[["young"]]$totCounts <- apply(X=GetAssayData(splatter_objs[["young"]], layer="counts"),MARGIN = 2, FUN=

TPcountDf[["young"]]$FNoverAvgSim <- TPcountDf[["young"]]$value / TPcountDf[["young"]]$avgExpr

TPcountDf[["young"]]$normOverSim <- TPcountDf[["young"]]$value / TPcountDf[["young"]]$totCounts * 10000

```

```

In [23]: FNcountDf[["young"]] %>%
  group_by(tool) %>%
  summarise(
    median_value = median(value, na.rm = TRUE),
    median_avgExpr = median(avgExpr, na.rm = TRUE),
    median_FNoverAvgSim = median(FNoverAvgSim, na.rm = TRUE),
    median_normOverSim = median(normOverSim, na.rm = TRUE)
  )

```

A tibble: 7 × 5

tool	median_value	median_avgExpr	median_FNoverAvgSim	median_normOverSim
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
STARsolo_TE	8	6	1.333333	1.364381
STARsolo_TE_EM	7	6	1.200000	1.200967
SoloTE_thr0	8	6	1.285714	1.295945
SoloTE_thr1	8	6	1.333333	1.328752
SoloTE_thr2	8	6	1.333333	1.346046
SoloTE_unique	8	6	1.375000	1.378360
Stellarscope	8	6	1.285714	1.309373

```
In [24]: TPcountDf[["young"]] %>%
  group_by(tool) %>%
  summarise(
    median_value = median(value, na.rm = TRUE),
    median_avgExpr = median(avgExpr, na.rm = TRUE),
    median_FNoverAvgSim = median(FNoverAvgSim, na.rm = TRUE),
    median_normOverSim = median(normOverSim, na.rm = TRUE)
  )
```

A tibble: 7 × 5

tool	median_value	median_avgExpr	median_FNoverAvgSim	median_normOverSim
<chr>	<dbl>	<dbl>	<dbl>	<dbl>
STARsolo_TE	10	6	1.666667	1.692477
STARsolo_TE_EM	10	6	1.714286	1.726035
SoloTE_thr0	10	6	1.625000	1.648879
SoloTE_thr1	11	6	1.833333	1.852119
SoloTE_thr2	12	6	2.000000	1.956800
SoloTE_unique	12	6	2.000000	2.075612
Stellarscope	10	6	1.666667	1.676767

```
In [25]: options(repr.plot.width=7, repr.plot.height=4.5)

#for(sample in samples){
  sample <- "young"
  #reorder tools
  FNcountDf[[sample]]$tool <- factor(FNcountDf[[sample]]$tool, levels=rev(names(colorTools)))

  show(ggplot(FNcountDf[[sample]], aes(y=tool, x=value, fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Raw counts of FN ", sample, " TEs")) +
    coord_trans(x = "sqrt") +
    guides(fill="none") +
    theme(text=element_text(size=20),
    # plot.title = element_text(size=18, hjust=0.5),
    # plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=10),
    # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    #panel.spacing = unit(1, "lines")

    ))

  show(ggplot(FNcountDf[[sample]], aes(y=tool, x=log2(value+1), fill=tool)) +
    geom_boxplot(outlier.size=0.5, linewidth=0.7) +
    scale_fill_manual(values=colorTools) +
    theme_pubclean() + ggtitle(paste0("Raw counts of FN ", sample, " TEs")) +
    guides(fill="none") +
    xlab("log2 count") +
    theme(text=element_text(size=20),
    # plot.title = element_text(size=18, hjust=0.5),
    # plot.subtitle = element_text(size=18, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=10),
    # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
    #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
    #panel.spacing = unit(1, "lines")
```

```

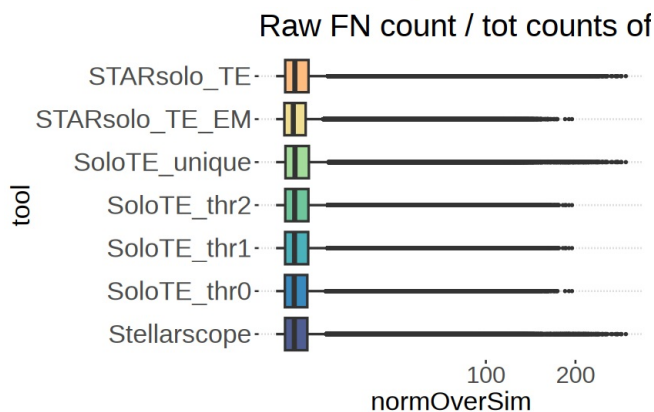
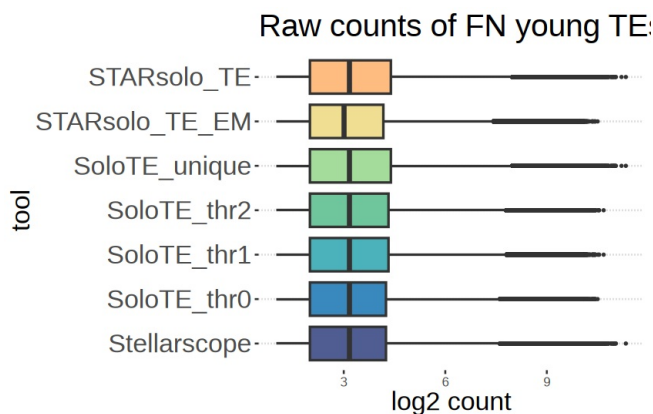
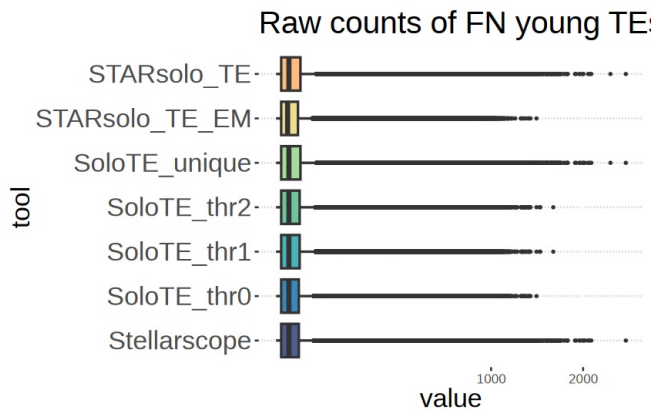
))

show(ggplot(FNcountDf[[sample]], aes(y=tool, x=normOverSim, fill=tool)) +
  geom_boxplot(outlier.size=0.5, linewidth=0.7) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Raw FN count / tot counts of ", sample, " TEs")) +
  guides(fill="none") +
  coord_trans(x = "sqrt") +
  theme(text=element_text(size=20),
  # plot.title = element_text(size=18, hjust=0.5),
  # plot.subtitle = element_text(size=18, hjust=0.5),
  axis.text.y = element_text(size=20),
  axis.text.x = element_text(size=18),
  # strip.text = element_text(size=17, hjust=0.5, vjust = 0.5),
  #strip.background = element_rect(fill="#F4F1EE", linewidth = 3, color = "white"),
  #panel.spacing = unit(1, "lines")

))

# ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FN_expression.pdf"), device="pdf", width=7, h
#}

```



```

In [28]: options(repr.plot.width=8, repr.plot.height=6.5)

#for(sample in samples){
  sample <- "young"
  #reorder tools
  FNvsTPcountDf[[sample]]$tool <- factor(FNvsTPcountDf[[sample]]$tool, levels=rev(names(colorTools)))

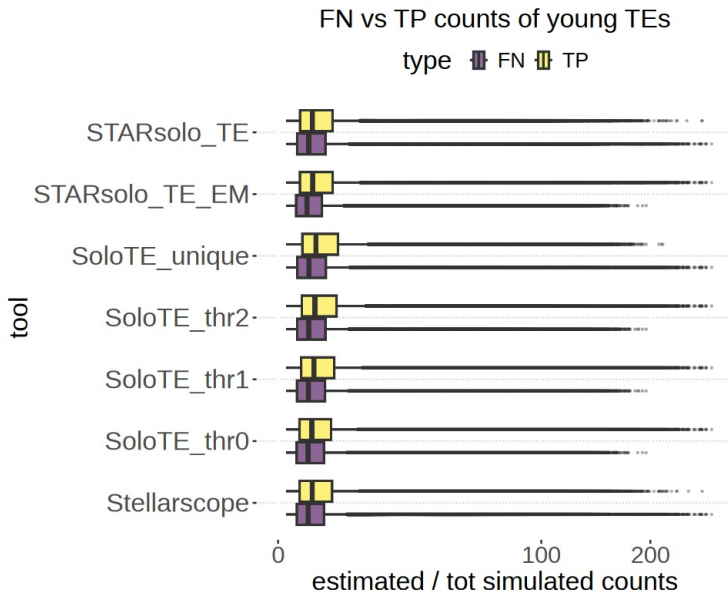
  show(ggplot(FNvsTPcountDf[[sample]], aes(y=tool, x=normOverSim, fill=type)) +
    rasterise(
      geom_boxplot(outlier.size = 0.4, outlier.alpha = 0.3, linewidth=0.7, alpha=0.6),
      dpi = 300 # Adjust DPI if you need higher/lower print resolution
    ) +

```

```

scale_fill_viridis_d()+
xlim(c(0, max(FNvsTPcountDf[[sample]]$normOverSim))) +
theme_pubclean() + ggtitle(paste0("FN vs TP counts of ", sample, " TEs")) +
coord_trans(x = "sqrt") +
xlab("estimated / tot simulated counts") +
theme(text=element_text(size=20),
      plot.title = element_text(size=20, hjust=0.5),
      plot.subtitle = element_text(size=20, hjust=0.5),
      axis.text.y = element_text(size=20),
      axis.text.x = element_text(size=18))
))
ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FN_vs TP_expression.png"), device="png", width=
ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FN_vs TP_expression.pdf"), device="pdf", width=8
#}

```



In [27]:

```
gc()
```

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	47055989	2513.1	71150768	3799.9	56579140	3021.7
Vcells	1913451968	14598.5	3328846661	25397.1	3328846661	25397.1

```

In [ ]: df_cell_avg <- FNvsTPcountDf[[sample]] %>%
  group_by(tool, cell, type) %>%
  summarise(
    avg_count = mean(value, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  pivot_wider(
    names_from = type,
    values_from = avg_count
  ) %>%
  filter(!is.na(TP), !is.na(FN)) %>% # both must exist at cell level
  mutate(
    FN_TP_avg_ratio = FN / TP,
    log2_FC = log2((FN + 1e-6) / (TP + 1e-6))
  )

sample <- "young"
#reorder tools
df_cell_avg$tool <- factor(df_cell_avg$tool, levels=rev(names(colorTools)))

options(repr.plot.width=8, repr.plot.height=4.5)

show(ggplot(df_cell_avg, aes(y=tool, x=FN_TP_avg_ratio, fill=tool)) +
  geom_boxplot(outlier.size=0.5, linewidth=0.7, alpha = 0.6) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Fold-change of FN vs TP counts of ", sample, " TEs"), subtitle="per ce
  #coord_trans(x = "sqrt") +
  guides(fill="none") +
  theme(text=element_text(size=20),
        plot.title = element_text(size=20, hjust=0.5),
        plot.subtitle = element_text(size=20, hjust=0.5),
        axis.text.y = element_text(size=20),
        axis.text.x = element_text(size=18))
  ))

```

```

ggsave(paste0("figures_", dataset_id, "_", sample, "/boxplot_FoldChange_FN_vs_TP_expression.pdf"), device="pdf")

#violin
show(ggplot(df_cell_avg, aes(y=tool, x=FN_TP_avg_ratio, fill=tool)) +
  geom_violin(outlier.size=0.5, linewidth=0.7, alpha=0.6) +
  scale_fill_manual(values=colorTools) +
  theme_pubclean() + ggtitle(paste0("Fold-change of FN vs TP counts of ", sample, " TEs"), subtitle="per cell") +
  #coord_trans(x = "sqrt") +
  guides(fill="none") +
  theme(text=element_text(size=20),
    plot.title = element_text(size=20, hjust=0.5),
    plot.subtitle = element_text(size=20, hjust=0.5),
    axis.text.y = element_text(size=20),
    axis.text.x = element_text(size=18))
))

```

Warning message in geom_violin(outlier.size = 0.5, linewidth = 0.7, alpha = 0.6):
 "Ignoring unknown parameters: `outlier.size`"

